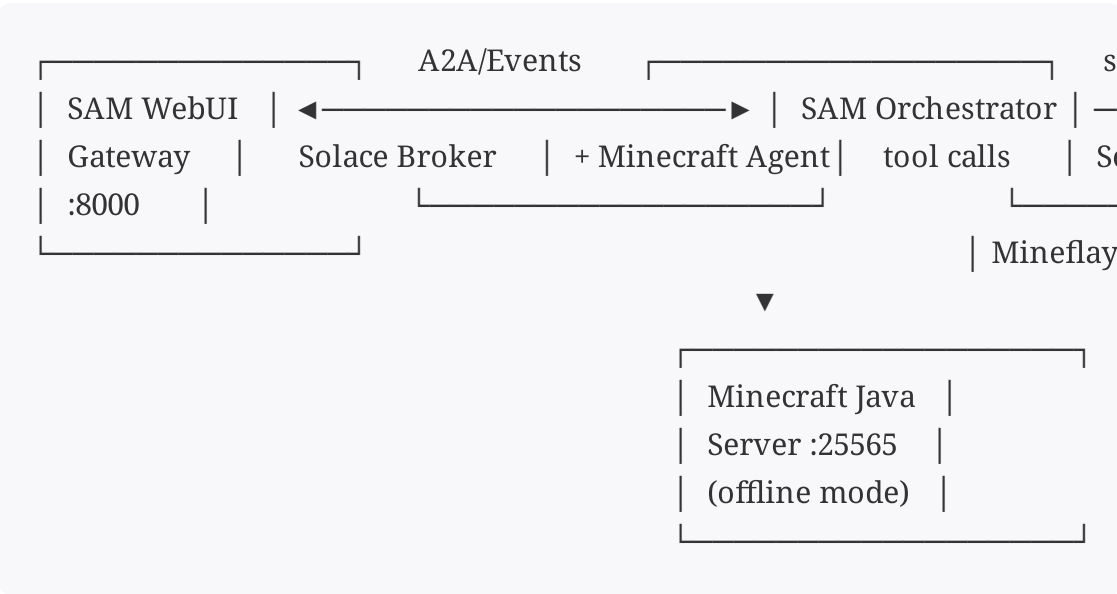


Solace Agent Mesh × Minecraft

MCP Demo Plan

Architecture Overview

The demo connects Solace Agent Mesh (SAM) to a Minecraft Java Edition server through a **Minecraft MCP Server** (built on Mineflayer). SAM's orchestrator decomposes natural language commands from the WebUI and delegates them to a Minecraft agent, which calls the MCP server's tools (move, build, dig, chat, etc.) via the stdio transport. The event broker handles all inter-agent communication using A2A protocol.



What You Need

Software Prerequisites

Component	Version / Details	Purpose
Python	3.11 or 3.12	SAM runtime [1]
Node.js	18+	Runs the Minecraft MCP server via npx [2]
Minecraft Java Edition	1.21.x (or match Mineflayer support)	Game server the bot connects to [2]
Solace Agent Mesh	Latest (pip install solace-agent-mesh)	Agent orchestration framework [1]
Minecraft MCP Server	github:yuniko-software/minecraft-mcp-server (recommended, 358★, actively maintained) or @gerred/mcpmc	MCP-compatible Minecraft bot controller [2][3]
LLM API access	OpenAI GPT-4o, Azure OpenAI, or Google Gemini	Powers agent reasoning [4]
Docker (optional)	Latest	Can run Minecraft server in container via itzg/minecraft-server [5]

Choosing a Minecraft MCP Server

Two main options exist:

- **yuniko-software/minecraft-mcp-server** — More feature-rich with 20+ commands (movement, flight, block placement, digging, inventory, entity interaction, chat, gamemode detection). Tested with Minecraft 1.21.8. Better for a visual demo.[2]
- **gerred/mcpmc** — Lighter, older package (29★), full MCP compatibility, TypeScript-based. Good if you want a minimal setup.[3]

Recommendation: Use yuniko-software/minecraft-mcp-server for the demo—it has richer tools for a more impressive showcase.

Available Minecraft MCP Tools

The Minecraft MCP server exposes the following tool categories that SAM agents can invoke:[2]

Movement: get-position, move-to-position, look-at, jump, move-in-direction

Flight: fly-to

Inventory: list-inventory, find-item, equip-item

Block Interaction: place-block, dig-block, get-block-info, find-block

Entity Interaction: find-entity

Communication: send-chat, read-chat

Game State: detect-gamemode

Step-by-Step Implementation Plan

Phase 1: Environment Setup

1. Set up Minecraft Server

The fastest approach is Docker with itzg/minecraft-server:[5]

```
# docker-compose.yml
version: "3.8"
services:
  mc:
    image: itzg/minecraft-server
    tty: true
    stdin_open: true
    ports:
      - "25565:25565"
    environment:
      TYPE: "VANILLA"
      VERSION: "1.21.4"
      EULA: "TRUE"
      MEMORY: "2G"
```

```
    ONLINE_MODE: "FALSE" # Required for Mineflayer bot connection
  volumes:
    - mc-data:/data
  volumes:
    mc-data:
```

Setting `ONLINE_MODE: FALSE` is critical—Mineflayer bots need offline mode to connect without Mojang authentication.[6][7]

Alternatively, open a singleplayer world to LAN (ESC → Open to LAN). [2]

2. Verify Node.js and npx

```
node --version # Should be 18+
npx --version
```

3. Test the Minecraft MCP Server standalone

Ensure the bot can connect before integrating with SAM:

```
npx -y github:yuniko-software/minecraft-mcp-server --host localhost --port 25565
```

You should see the bot join in-game.

Phase 2: SAM Project Initialization

1. Create and set up the SAM project

```
mkdir sam-minecraft-demo && cd sam-minecraft-demo
python3 -m venv venv
source venv/bin/activate
pip install solace-agent-mesh
sam init --gui
```

In the GUI, choose "Get Started Quickly" for the initial setup. Configure your LLM provider (OpenAI, Azure, or Gemini).[8][4]

2. Verify base installation

```
sam run
```

Navigate to <http://127.0.0.1:8000/> and confirm the WebUI and orchestrator work.[8]

Phase 3: Create the Minecraft Agent

Option A: Via GUI

```
sam add agent --gui
```

In the GUI:

1. Name: MinecraftAgent, Display Name: Minecraft Bot Controller
2. Add an **MCP Tool** with connection params:[9][8]

```
{  
  "type": "stdio",  
  "command": "npx",  
  "args": ["-y", "github:yuniko-software/minecraft-mcp-server", "--host",  
}
```

3. Also add the artifact_management builtin-group
4. Set the Agent Card description
5. Save

Option B: Manual YAML (recommended for fine-tuning)

Create configs/agents/minecraft_agent.yaml:

```
log:  
  stdout_log_level: INFO  
  log_file_level: DEBUG  
  log_file: minecraft_agent.log  
  
!include ../shared_config.yaml
```

apps:

```
- name: minecraft_agent_app
  app_base_path: .
  app_module: solace_agent_mesh.agent.sac.app
  broker:
    <<: *broker_connection
```

app_config:

```
namespace: "${NAMESPACE}"
supports_streaming: true
agent_name: "MinecraftAgent"
display_name: "Minecraft Bot"
model: *planning_model
instruction: |
```

You are a Minecraft bot controller agent. You control a bot inside a Mine

You can perform the following actions:

- Move to specific coordinates or in a direction
- Fly to locations (if in creative mode)
- Place and dig blocks at specific coordinates
- Check inventory, find items, and equip them
- Find nearby blocks and entities
- Send chat messages in-game and read chat history
- Detect the current gamemode

When asked to build something, break it down into individual block pla

When navigating, use get-position first to understand where you are, th

Always confirm actions by reporting back what happened.

tools:

Minecraft MCP Server

```
- tool_type: mcp
  connection_params:
    type: stdio
    command: "npx"
    args:
      - "-y"
```

- "github:yuniko-software/minecraft-mcp-server"
- "--host"
- "localhost"
- "--port"
- "25565"
- "--username"
- "SAMBot"

timeout: 60

Built-in artifact tools

- tool_type: builtin-group
- group_name: "artifact_management"

session_service:

type: "memory"

default_behavior: "PERSISTENT"

artifact_service:

type: "filesystem"

base_path: "/tmp/samv2"

artifact_scope: namespace

artifact_handling_mode: "reference"

enable_embed_resolution: true

enable_artifact_content_instruction: true

Agent Card (A2A protocol)

agent_card:

description: |

An agent that controls a Minecraft bot in real-time. It can navigate the place and break blocks, manage inventory, interact with entities, and c via in-game chat. Use this agent for any Minecraft-related actions.

defaultInputModes: [text]

defaultOutputModes: [text]

skills:

- id: "minecraft_movement"

name: "Movement & Navigation"

description: "Move the bot to coordinates, look at positions, jump, or

- id: "minecraft_building"

```
  name: "Block Manipulation"
  description: "Place blocks, dig blocks, find blocks, get block info"
- id: "minecraft_inventory"
  name: "Inventory Management"
  description: "List inventory, find items, equip items"
- id: "minecraft_communication"
  name: "In-Game Chat"
  description: "Send and read chat messages in the Minecraft world"
```

```
# Discovery & Communication
```

```
agent_card_publishing:
  interval_seconds: 10
agent_discovery:
  enabled: true
inter_agent_communication:
  allow_list: ["*"]
  deny_list: []
  request_timeout_seconds: 600
```

This YAML configuration follows the exact patterns from the official SAM MCP integration tutorial, adapted for the Minecraft MCP server. [9][8]

Phase 4: Extend with Additional Agents (Optional but recommended for demo impact)

To make the demo more impressive, add complementary agents:

1. Architect Agent (Custom Python tool)

A planning agent that generates build blueprints (coordinate lists) for structures. It doesn't touch Minecraft directly—it delegates to the `MinecraftAgent`.

```
# configs/agents/architect_agent.yaml
# ... (standard boilerplate)
app_config:
  agent_name: "ArchitectAgent"
```



```
display_name: "Minecraft Architect"
model: *planning_model
instruction: |
    You are a Minecraft architect. When asked to design a structure, you produce
    a list of block placements with coordinates and block types. You then delegate
    the actual placement to the MinecraftAgent via inter-agent communication
    tools:
    - tool_type: builtin-group
      group_name: "artifact_management"
agent_card:
    description: "Plans and designs Minecraft structures, then coordinates with
    defaultInputModes: [text]
    defaultOutputModes: [text]
```

2. Chat Monitor Agent

An agent that reads Minecraft in-game chat and responds to player commands.

Phase 5: Run and Demo

1. Start all components

```
# Terminal 1: Minecraft server
docker compose up -d

# Terminal 2: SAM
sam run
```

2. Demo scenarios to showcase

Open the WebUI at `http://localhost:8000/` and try:

Prompt	What It Demonstrates
"Where is the bot right now in Minecraft?"	Basic tool call (get-position)
"Move to coordinates 100, 64, 100"	Navigation (move-to-position)
"Build a 3x3 stone platform at my current location"	Multi-step block placement loop
"What's in the bot's inventory?"	Inventory management
"Find the nearest tree and chop it down"	Block finding + digging sequence
"Send a hello message in the Minecraft chat"	In-game communication
"Build a small house with walls and a door"	Complex multi-agent workflow (if Architect agent is added)

3. What to highlight in the demo

- **Orchestrator decomposition:** Show the Agent Workflow view in WebUI to visualize how the orchestrator routes tasks[8]
- **MCP tool normalization:** One agent combining MCP tools with built-in SAM tools[10][9]
- **A2A protocol:** If using multiple agents, show inter-agent delegation
- **Event-driven backbone:** All communication flows through the Solace broker, providing observability

Potential Challenges and Solutions

Challenge	Solution
Minecraft version mismatch with Mineflayer	Pin VERSION in Docker compose to a Mineflayer-supported version (1.21.x)
Bot fails to connect	Ensure ONLINE_MODE=FALSE on the server [6]
MCP server timeout on complex builds	Increase timeout in the YAML connection_params (default 30s → 120s)
LLM generates invalid coordinates	Add coordinate validation in the agent instruction prompt
npx fails to resolve package	Pre-install globally: npm install -g github:yuniko-software/minecraft-mcp-server
Bot gets stuck on terrain	Use fly-to in creative mode for demo reliability

Recommended Demo Mode

Set the Minecraft server to **Creative mode** (gamemode creative) for the demo. This gives the bot unlimited blocks and flight capability, making the demo more visually impressive and avoiding survival-mode complications like health/hunger.[2]

```
# Via RCON or server console
/gamemode creative SAMBot
```

Summary Checklist

- [] Minecraft Java server running (Docker or local) with online-mode=false
- [] Node.js 18+ installed with npx available
- [] Python 3.11+ virtual environment with solace-agent-mesh installed

- [] SAM project initialized with LLM configured
 - [] minecraft_agent.yaml created with MCP stdio connection to Minecraft MCP server
 - [] sam run starts successfully, bot joins Minecraft world
 - [] Test basic commands: position, movement, block placement
 - [] (Optional) Add Architect agent for multi-agent workflow
 - [] Prepare demo script with escalating complexity
-

References

1. [solace-agent-mesh](#) - Integrating a Model Context Protocol (MCP) Server into Solace Agent Mesh. ~10–15 min, MCP Integratio...
2. [yuniko-software/minecraft-mcp-server](#) - A Minecraft MCP Server powered by Mineflayer API. It allows to control a Minecraft character in real...
3. [gerred/mcpmc: Model Context Protocol Minecraft Server](#) - A Model Context Protocol (MCP) server for interacting with Minecraft via Mineflayer. This package en...
4. [Orchestrate AI Agents with Solace Agent Mesh | Adaptiv AU](#) - Learn Solace Agent Mesh and build your first multi-agent workflow, using an IT ticket example to red...
5. [How to Run a Minecraft Java Server using Docker](#) - In this guide you will be learning how to run a Minecraft Java Edition server using Docker. Easily m...
6. [Tutorial for Making Mineflayer Bots](#) - This week we're coding a simple Minecraft bot using Mineflayer. Details for nerds: The bot is writte...
7. [Mineflayer – Create Advanced Minecraft Bots Easily](#) - Does Mineflayer work with modded Minecraft servers? ... Mineflayer works with both premium and offli...
8. [Getting started with Solace Agent Mesh](#) - By the end of this codelab, you'll be equipped to get started with Solace Agent Mesh, including lear...
9. [MCP + Solace Agent Mesh: The Deep Dive](#) - To learn more about how MCP is integrated with Solace Agent Mesh read more here: MCP Integration | S...

10. [The Anatomy of Agents in Solace Agent Mesh](#) - Tune in for the next blog to dive deeper into MCP configurations for agents, where we'll explore adv...